

MCT-454L Mobile Robotics

Lab Manual 6: Reactive Navigation Using Turtlebot3 LiDAR in Gazebo Environment

1. Lab Objective

This lab focuses on using LiDAR data to generate motion commands for reactive navigation.

In this session, you will use the `/scan` topic to perceive the environment and implement behaviors such as obstacle detection, avoidance, and wall following.

2. Learning Outcomes

After completing this lab, you will be able to:

- Interpret LaserScan data in ROS 2
- Extract directional distance information (front, left, right)
- Compute motion commands based on sensor data
- Implement reactive navigation without maps

3. Pre-Lab Setup

Launch simulation

```
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Verify LiDAR topic

```
ros2 topic list
ros2 topic echo /scan
```

Visualize scan (optional)

```
ros2 run rviz2 rviz2
```

4. Understanding /scan Topic

This section helps interpret LiDAR data structure.

The `/scan` topic publishes:

- `ranges[]`: array of distances
- `angle_min, angle_max`: scan range
- `angle_increment`: angular resolution

Each index corresponds to a direction around the robot.

5. Lab Tasks

Task 1: Scan Processing

Objective: Extract directional distances.

- Subscribe to `/scan`
- Divide scan into regions:
 - Front
 - Left
 - Right
- Compute minimum distance in each region

Task 2: Stop-on-Obstacle

Objective: Implement safety behavior.

- Move forward
- Stop when front distance $<$ threshold

Task 3: Obstacle Avoidance

Objective: Generate motion decisions.

- If front is blocked $\rightarrow$ turn
- Turn toward side with larger clearance

Task 4: Wall Following

Objective: Maintain fixed distance from wall.

- Use left or right scan
- Apply proportional control

Task 5: Behavior Sequencing

Objective: Combine behaviors.

Sequence example:

- Move forward
- Avoid obstacle
- Follow wall in corridor

6. Implementation Guidelines

- Publish to `/cmd_vel`
- Suggested speeds:
 - Linear: 0.1–0.2 m/s
 - Angular: 0.3–0.8 rad/s
- Handle invalid values (`inf`, `NaN`)
- Use averaging or min filtering for stability

7. Boilerplate ROS 2 Node (Students Fill In)

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import Twist
import numpy as np

class LidarNavigator(Node):

    def __init__(self):
        super().__init__('lidar_navigator')

        self.subscription = self.create_subscription(
            LaserScan,
            '/scan',
            self.scan_callback,
            10)

        self.publisher = self.create_publisher(Twist, '/cmd_vel', 10)

        # TODO: Define thresholds
        self.front_threshold = _____
        self.side_threshold = _____

    def scan_callback(self, msg):
        ranges = np.array(msg.ranges)

        # -----
        # TODO 1: Clean data (remove inf/nan)
        # -----
        ranges = _____

        # -----
        # TODO 2: Define regions
        # -----
        front = _____
        left = _____
        right = _____

        # Compute minimum distance
        front_dist = _____
        left_dist = _____
        right_dist = _____

        twist = Twist()

        # -----
        # TODO 3: Obstacle logic
        # -----
        if _____: # obstacle in front
```

```

        # -----
        # TODO 4: Turn direction
        # -----
        if _____:    # left clearer
            twist.angular.z = _____
        else:
            twist.angular.z = _____

        twist.linear.x = _____

    else:
        # -----
        # TODO 5: Forward motion
        # -----
        twist.linear.x = _____
        twist.angular.z = _____

    self.publisher.publish(twist)

def main(args=None):
    rclpy.init(args=args)
    node = LidarNavigator()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

8. Helpful Commands (Hints)

Check scan values

```
ros2 topic echo /scan
```

Monitor velocity commands

```
ros2 topic echo /cmd_vel
```

Debug printing (inside code)

```
self.get_logger().info(f"Front: {front_dist:.2f}")
```

10. Expected Demonstration

Students must demonstrate:

- Robot stops at obstacle
- Robot avoids obstacle
- Robot follows wall
- Robot navigates without collision

11. Conclusion

This lab session provided hands-on experience in implementing reactive navigation using LiDAR data from the `/scan` topic. Students processed range measurements to extract directional information and used it to generate motion commands for obstacle detection and avoidance.

Through this exercise, students developed an understanding of how raw sensor data can be converted into meaningful control decisions in real time. The lab also highlighted the importance of parameter tuning, handling noisy sensor data, and designing stable control logic.

These concepts form a foundation for more advanced topics such as autonomous navigation, path planning, and sensor fusion in mobile robotics.

Deliverables

Students are required to submit the following deliverables at the end of the lab session:

1. Create a ROS 2 package implementing the LiDAR-based navigation node and push the complete package to a **GitHub repository**.
2. Provide the **source code** for the LiDAR navigation node. The code must include:
 - Scan processing logic
 - Region extraction (front, left, right)
 - Decision-making for obstacle avoidance
3. Include **screenshots** showing:
 - LaserScan visualization
 - Robot navigating in the environment
4. Include a screenshot of **rqt_graph** showing:
 - `/scan` subscriber
 - `/cmd_vel` publisher
 - active ROS 2 nodes
5. Demonstrate the following behaviors and include **screenshots or short video clips**:
 - Robot stopping at an obstacle
 - Robot avoiding an obstacle
 - Robot navigating without collision
6. Provide **observations** on:
 - Robot behavior near obstacles
 - Any oscillations or instability
 - Effect of changing threshold values
7. Write a brief **conclusion** summarizing the learning outcomes and challenges faced.